DSA5208: Scalable Distributed Computing for Data Science

Lecture 2: Broadcast and Synchronizers

Zhenning Cai

Department of Mathematics, National University of Singapore

AY2026/2027, Semester 1

1 Broadcast

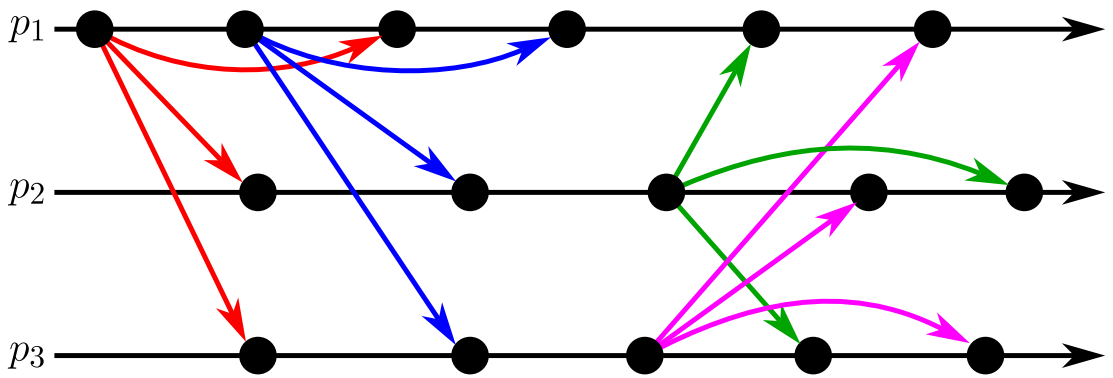
2 Shortest-path problem

3 Synchronizers

4 Summary

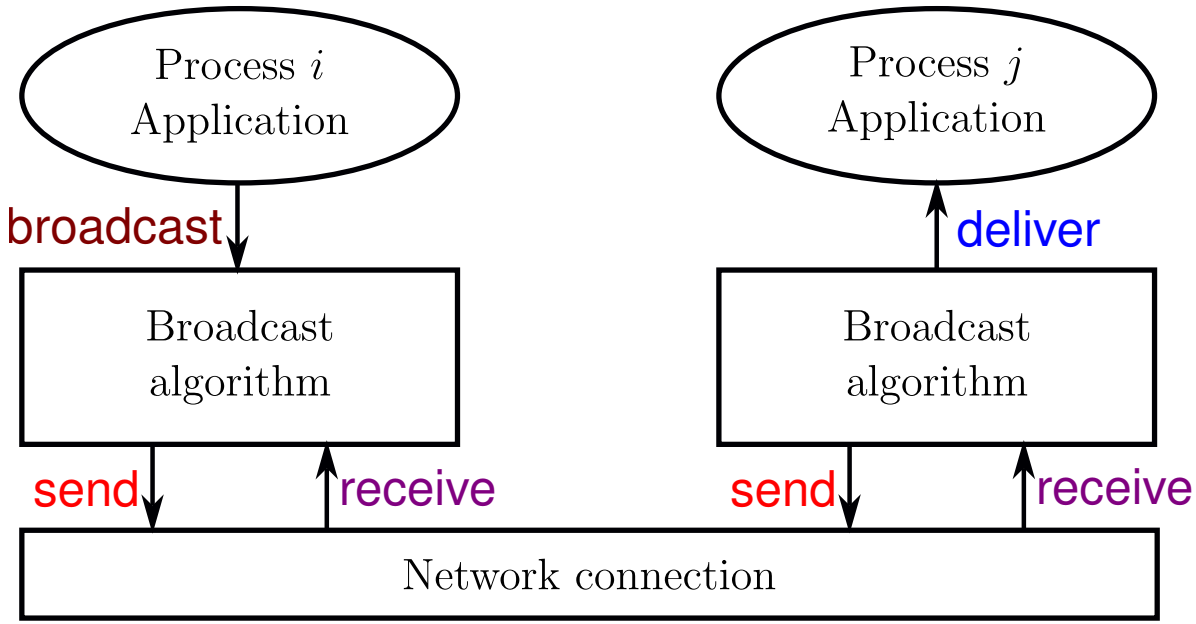
Broadcast protocols

Broadcast: A process p_i sends the same message to all processes p_j, $j = 1, \cdots, n$.



If there are many messages being broadcast in the system, in which order will they be delivered?

Receiving versus delivering



After the broadcast algorithm receives message from the network, it may hold it in the buffer for a while before delivering the message to the application.

Broadcast models

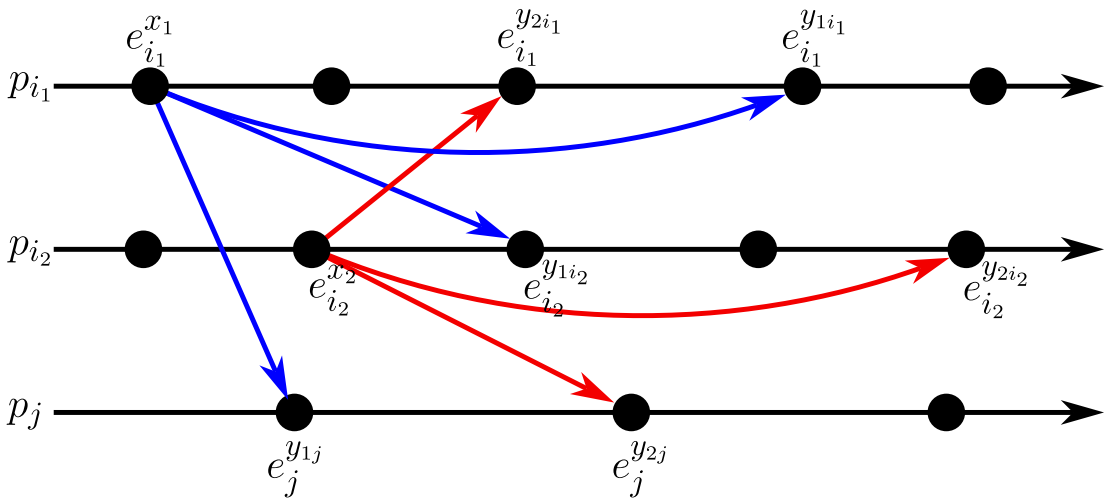
Assume

- $e_{i_1}^{x_1}$ broadcasts the message m_1, and $e_{i_2}^{x_2}$ broadcasts the message m_2.
- m_1 is delivered to process j in the event $e_j^{y_{1j}}$, and m_2 is delivered to process j in the event $e_j^{y_{2j}}$.

Broadcast models

Assume

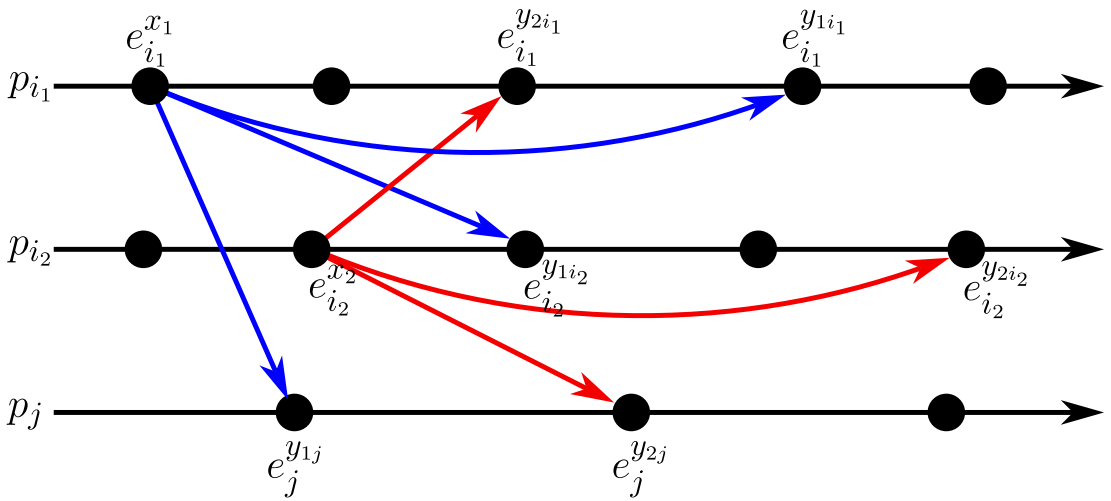
- $e_{i_1}^{x_1}$ broadcasts the message m_1, and $e_{i_2}^{x_2}$ broadcasts the message m_2.
- m_1 is delivered to process j in the event $e_j^{y_{1j}}$, and m_2 is delivered to process j in the event $e_j^{y_{2j}}$.



Broadcast models

Assume

- $e_{i_1}^{x_1}$ broadcasts the message m_1, and $e_{i_2}^{x_2}$ broadcasts the message m_2.
- m_1 is delivered to process j in the event $e_j^{y_{1j}}$, and m_2 is delivered to process j in the event $e_j^{y_{2j}}$.



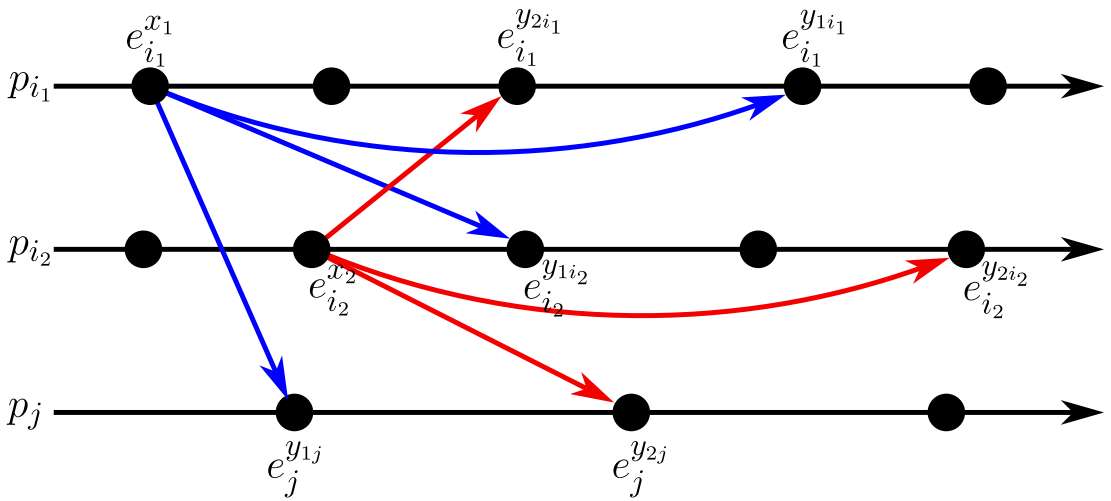
Broadcast models:

- **FIFO broadcast:** $i_1 = i_2$ and $e_{i_1}^{x_1} \rightarrow e_{i_2}^{x_2} \implies e_j^{y_{1j}} \rightarrow e_j^{y_{2j}}, \forall j.$

Broadcast models

Assume

- $e_{i_1}^{x_1}$ broadcasts the message m_1, and $e_{i_2}^{x_2}$ broadcasts the message m_2.
- m_1 is delivered to process j in the event $e_j^{y_{1j}}$, and m_2 is delivered to process j in the event $e_j^{y_{2j}}$.



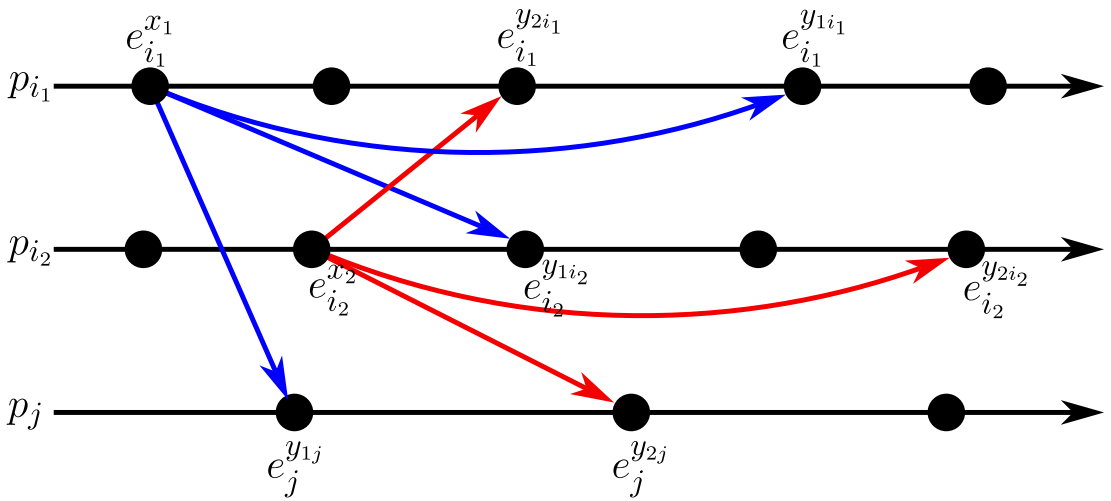
Broadcast models:

- **Causal broadcast:** $e_{i_1}^{x_1} \rightarrow e_{i_2}^{x_2} \implies e_j^{y_{1j}} \rightarrow e_j^{y_{2j}}, \forall j.$

Broadcast models

Assume

- $e_{i_1}^{x_1}$ broadcasts the message m_1, and $e_{i_2}^{x_2}$ broadcasts the message m_2.
- m_1 is delivered to process j in the event $e_j^{y_{1j}}$, and m_2 is delivered to process j in the event $e_j^{y_{2j}}$.



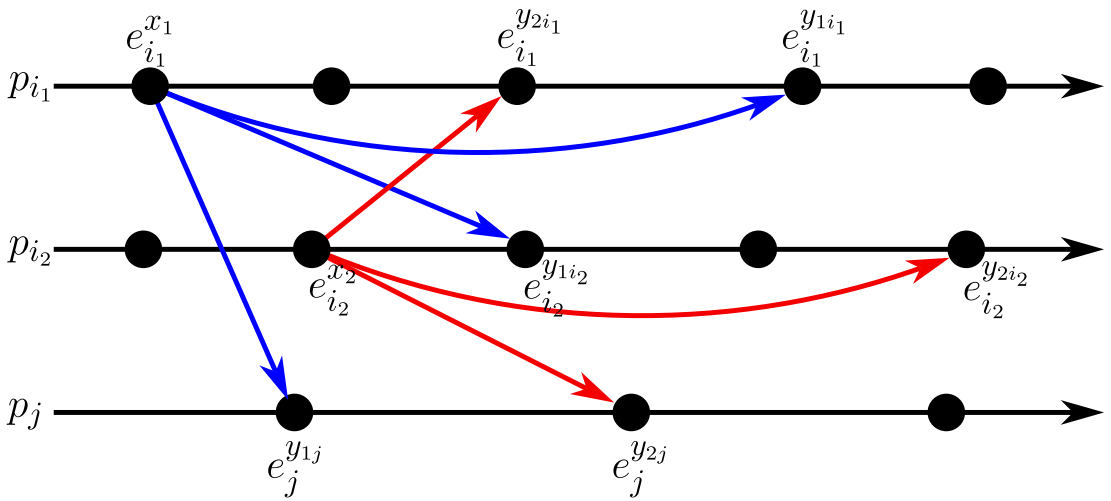
Broadcast models:

- **Total order broadcast:** either $e_j^{y_{1j}} \rightarrow e_j^{y_{2j}}$, $\forall j$ or $e_j^{y_{2j}} \rightarrow e_j^{y_{1j}}$, $\forall j$.

Broadcast models

Assume

- $e_{i_1}^{x_1}$ broadcasts the message m_1, and $e_{i_2}^{x_2}$ broadcasts the message m_2.
- m_1 is delivered to process j in the event $e_j^{y_{1j}}$, and m_2 is delivered to process j in the event $e_j^{y_{2j}}$.

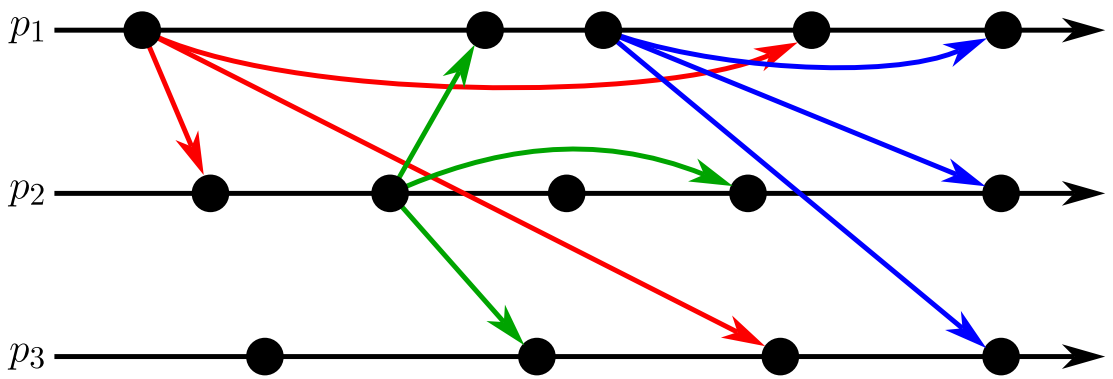


Broadcast models:

- **FIFO-Total order broadcast:** Combination of FIFO broadcast and Total order broadcast.

FIFO broadcast

If two messages m_1 and m_2 are broadcast by the same process, and m_1 is broadcast before m_2, then on any process, m_1 is delivered before m_2.



- Messages sent by different processes can be delivered in any order.

Implementation of FIFO broadcast

Algorithm FIFO broadcast

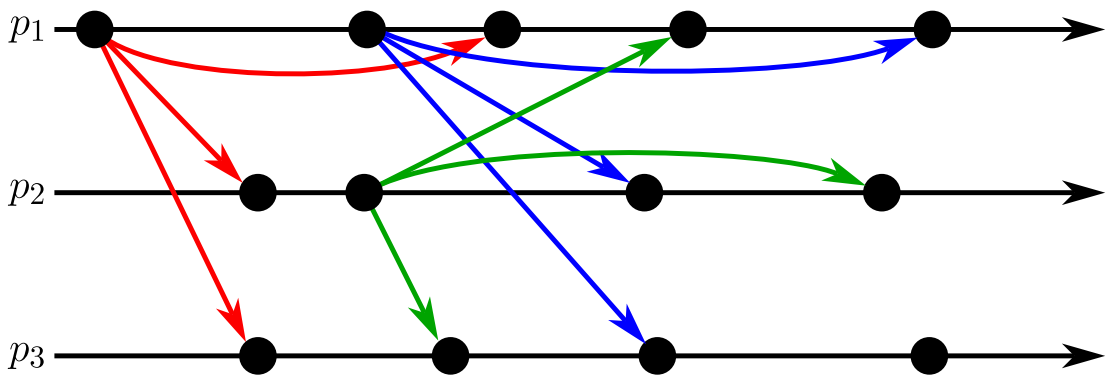
(Local variables)
$sendSeq \leftarrow 0$
$delivered[1 \cdots n] \leftarrow 0$
$buffer \leftarrow \{\}$

function BROADCASTSEND(msg: message to be sent)
    Send $(i, sendSeq, msg)$ to all processes (i: sender index)
    $sendSeq := sendSeq + 1$
end function

function BROADCASTRECV((i, seq, msg): received message)
    $buffer := buffer \cup \{(i, seq, msg)\}$
    while $\exists sender, m$ such that $(sender, delivered[sender], m) \in buffer$ **do**
        Deliver the message m
        $buffer := buffer \backslash \{(sender, delivered[sender], m)\}$
        $delivered[sender] := delivered[sender] + 1$
    end while
end function

Causal broadcast

If the broadcast of m_1 happens before the broadcast of m_2, then m_1 is delivered before m_2 on all processes.



- Concurrent messages can be delivered in any order.
- Causal broadcast $\Longrightarrow$ FIFO broadcast.

Implementation of causal broadcast

Algorithm Causal broadcast

1: (Local variables)
2: $sendSeq \leftarrow 0$
3: $delivered[1 \cdots n] \leftarrow 0$
4: $buffer \leftarrow \{\}$

5: **function** BROADCASTSEND(msg: message to be sent)
6: $\quad deps := delivered$
7: $\quad deps[i] := sendSeq$ (i: sender index)
8: $\quad$ Send $(i, deps, msg)$ to all processes
9: $\quad sendSeq := sendSeq + 1$
10: **end function**

Implementation of causal broadcast

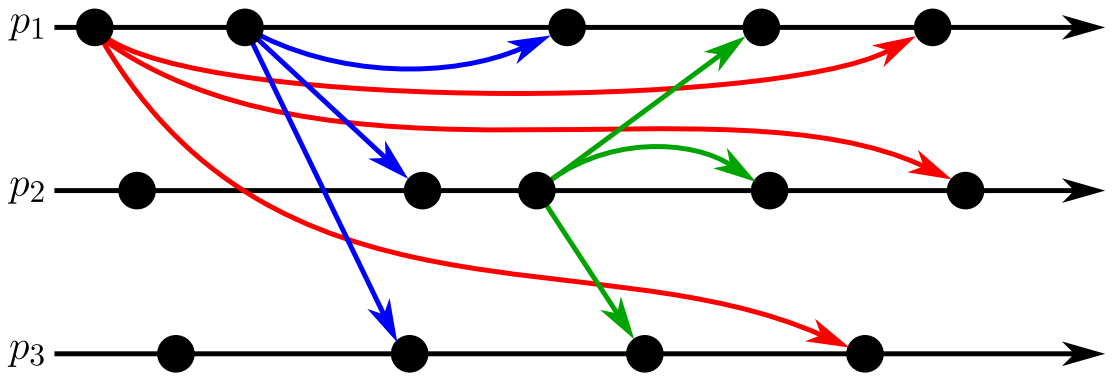
Algorithm Causal broadcast

1: (Local variables)
2: $sendSeq \leftarrow 0$
3: $delivered[1 \cdots n] \leftarrow 0$
4: $buffer \leftarrow \{\}$

11: **function** BROADCASTRECV((i, dep, msg): received message)
12: $\quad buffer := buffer \cup \{(i, dep, msg)\}$
13: $\quad$ **while** $\exists (sender, deps, m) \in buffer$ such that $deps \leq delivered$ **do**
14: $\quad\quad$ Deliver the message m
15: $\quad\quad buffer := buffer \backslash \{(sender, deps, m)\}$
16: $\quad\quad delivered[sender] := delivered[sender] + 1$
17: $\quad$ **end while**
18: **end function**

Total order broadcast

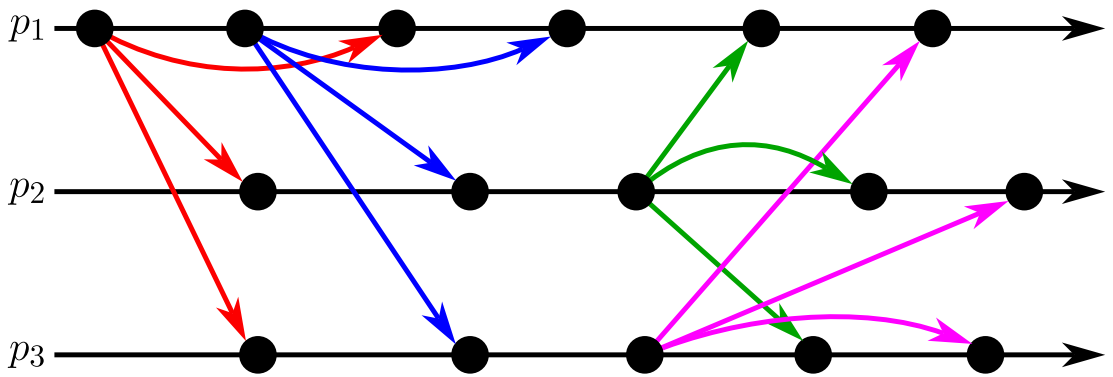
All processes deliver messages in the same order.



- If the message m_2 depends on the message m_1, meaning that m_2 is broadcast after m_1 is received on the same process, then the total order broadcast indicates that m_1 is delivered before m_2 on all processes.
- When m_1 and m_2 are broadcast by the same process, the delivery could be non-causal.

FIFO-total order broadcast

All messages are delivered in the same order, and messages broadcast by the same process follows the FIFO order.



- FIFO-total order broadcast $\Longrightarrow$ causal broadcast.

Implementation of FIFO-total order broadcast

FIFO-total order broadcast can be implemented using Lamport's scalar time (Skeen's algorithm):

- The sender broadcasts the message to all processes.
- Each recipient sends its local scalar time back to the sender and keeps the message on hold.
- The sender collects all the received timestamps and broadcasts the largest one to all recipients.
- Recipients order messages by this timestamp and deliver a message when it is first in the queue.

Implementation of FIFO-total order broadcast

FIFO-total order broadcast can be implemented using Lamport's scalar time (Skeen's algorithm):

- The sender broadcasts the message to all processes.
- Each recipient sends its local scalar time back to the sender and keeps the message on hold.
- The sender collects all the received timestamps and broadcasts the largest one to all recipients.
- Recipients order messages by this timestamp and deliver a message when it is first in the queue.

Alternatively, any process that wants to broadcast a message should send the message to a leader, and the leader will perform a FIFO broadcast of the message to all processes.

- The naive implementation of the algorithm is not fault tolerant.
- It needs to be supplemented by a leader-election algorithm.

1 Broadcast

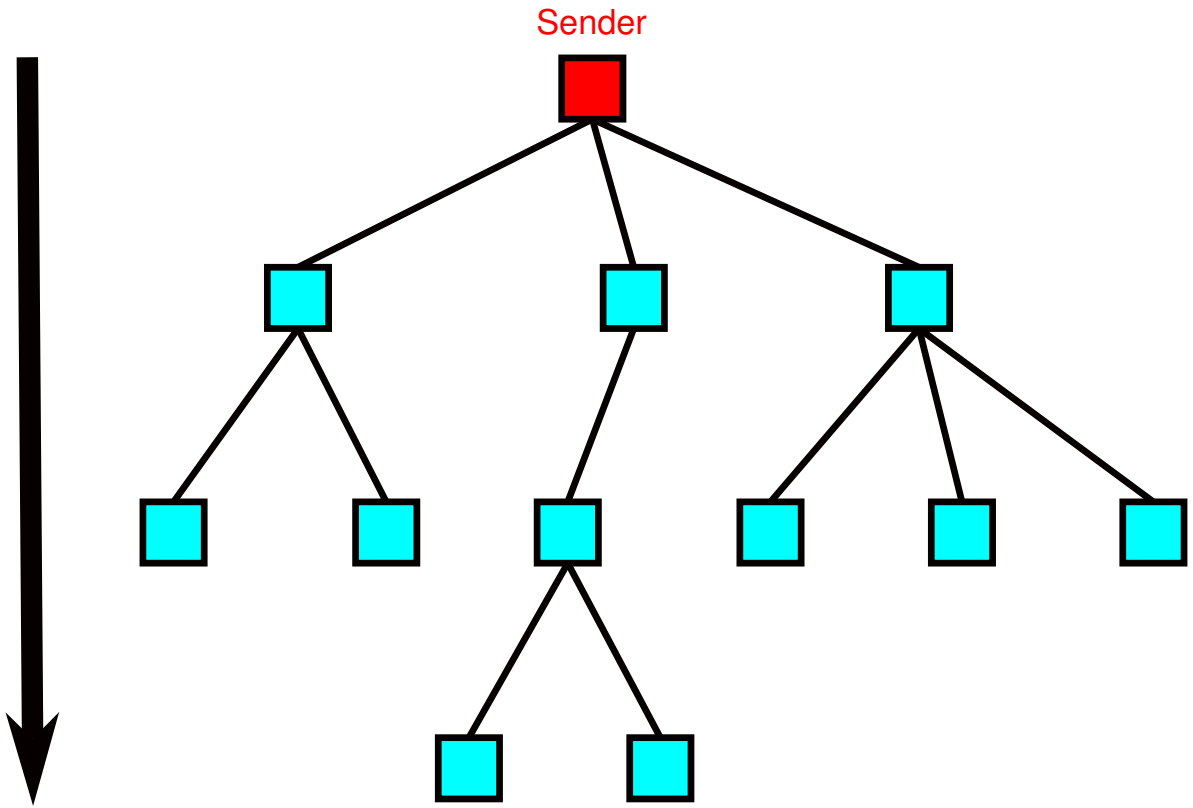
2 Shortest-path problem

3 Synchronizers

4 Summary

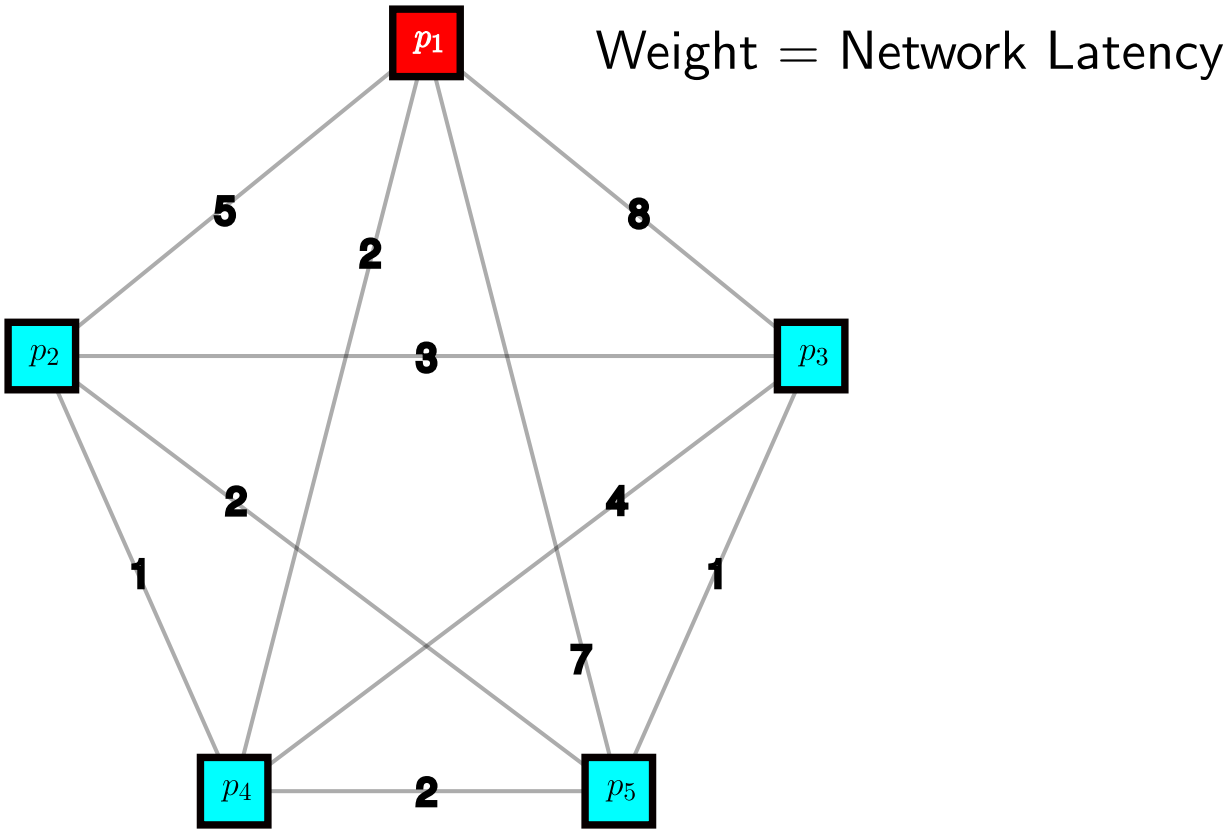
Broadcast on a tree

How is the message broadcast from the sender to all other processes?



Single source shortest path problem

Problem: Find the shortest path from the sender to all other processes.



Synchronous and asynchronous systems

- A synchronous system is a system that satisfies the following properties:
  1. There is a known upper bound on the message communication delay.
  2. There is a bounded drift rate for the local clock of each processor with respect to real time.
  3. There is a known upper bound on the time taken by a process to execute a logical step in the execution.

Synchronous and asynchronous systems

- A synchronous system is a system that satisfies the following properties:
  1. There is a known upper bound on the message communication delay.
  2. There is a bounded drift rate for the local clock of each processor with respect to real time.
  3. There is a known upper bound on the time taken by a process to execute a logical step in the execution.

  In a synchronous system, every process knows when other processes will complete certain executions.

Synchronous and asynchronous systems

- A synchronous system is a system that satisfies the following properties:
  1. There is a known upper bound on the message communication delay.
  2. There is a bounded drift rate for the local clock of each processor with respect to real time.
  3. There is a known upper bound on the time taken by a process to execute a logical step in the execution.

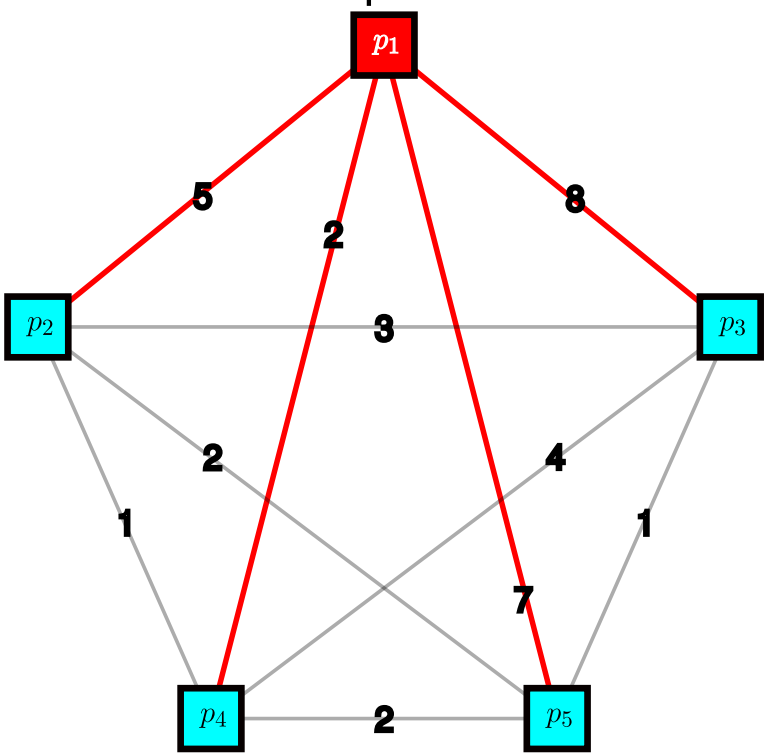
  In a synchronous system, every process knows when other processes will complete certain executions.
- An asynchronous system is a system in which none of the above properties are satisfied. Distributed systems are inherently asynchronous.

Synchronous Bellman-Ford algorithm

In each round,

$$(length\ of\ p_i) = \min_j\{(length\ of\ p_j) + weight_{j,i}\}.$$

The shortest paths are found after $n-1$ rounds.



Round 1:

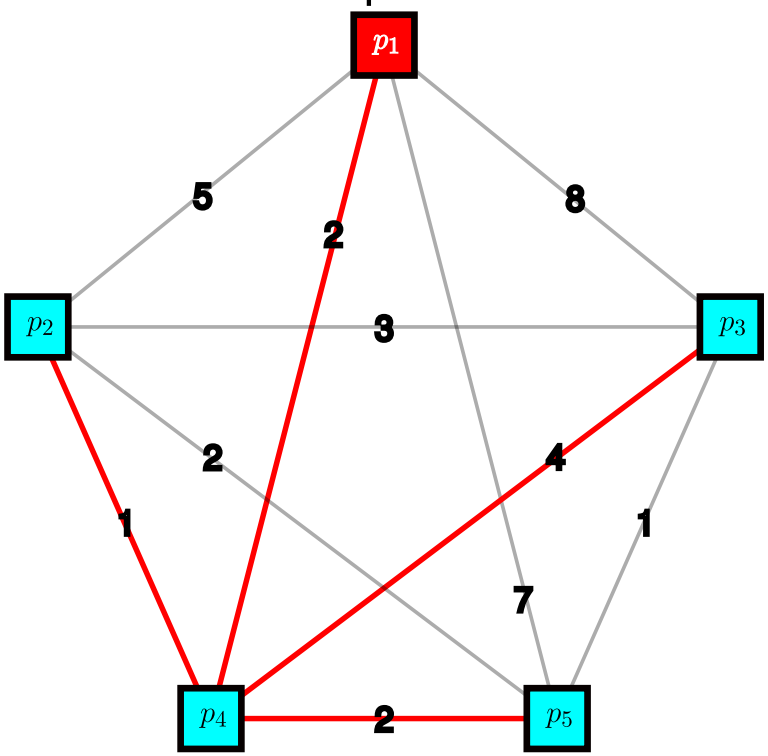
- p_1: $length = 0$, $parent = \perp$
- p_2: $length = 5$, $parent = 1$
- p_3: $length = 8$, $parent = 1$
- p_4: $length = 2$, $parent = 1$
- p_5: $length = 7$, $parent = 1$

Synchronous Bellman-Ford algorithm

In each round,

$$(length\ of\ p_i) = \min_j\{(length\ of\ p_j) + weight_{j,i}\}.$$

The shortest paths are found after $n-1$ rounds.



Round 2:

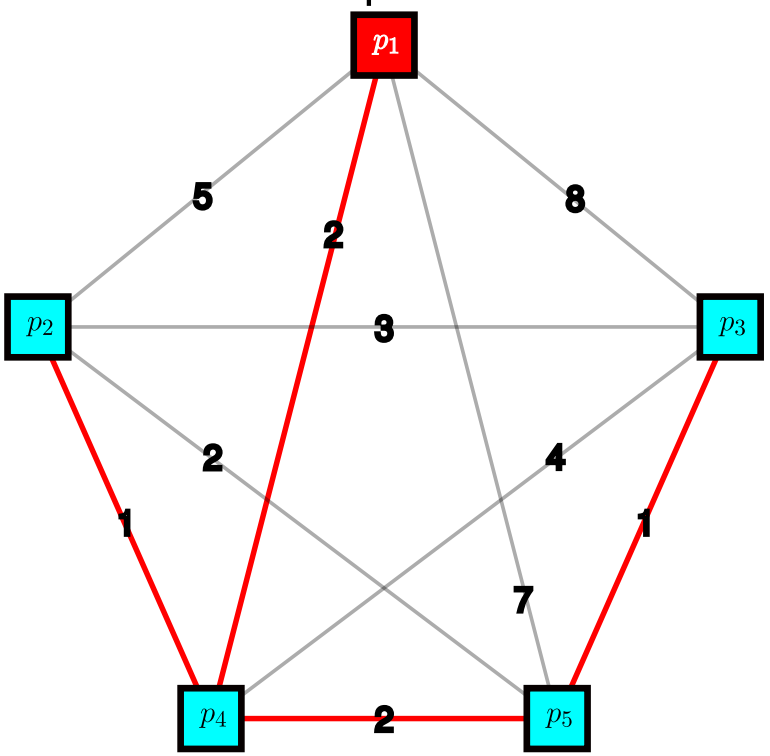
- p_1: $length = 0$, $parent = \perp$
- p_2: $length = 3$, $parent = 4$
- p_3: $length = 6$, $parent = 4$
- p_4: $length = 2$, $parent = 1$
- p_5: $length = 4$, $parent = 4$

Synchronous Bellman-Ford algorithm

In each round,

$$(length\ of\ p_i) = \min_j\{(length\ of\ p_j) + weight_{j,i}\}.$$

The shortest paths are found after $n-1$ rounds.



Round 3:

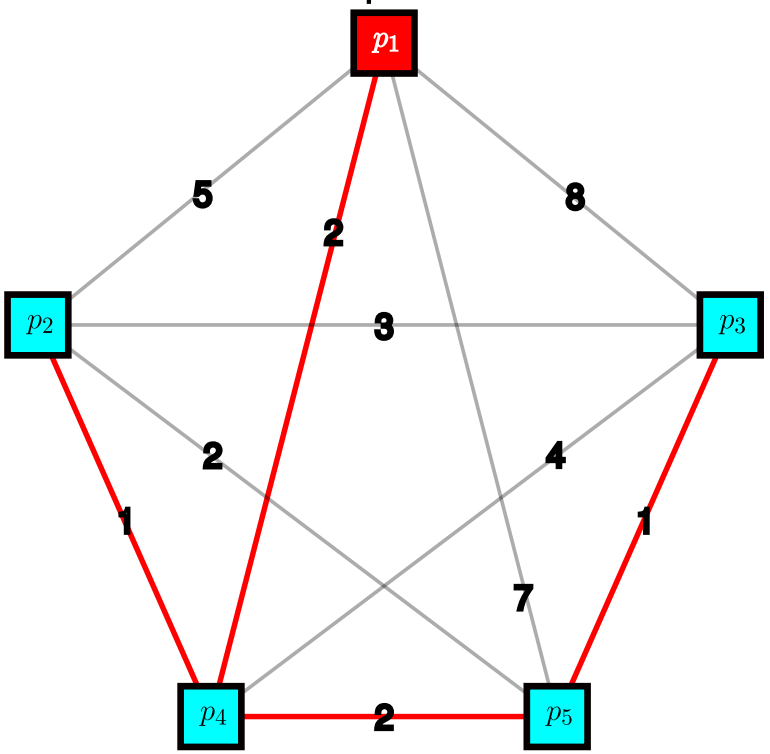
- p_1: $length = 0$, $parent = \perp$
- p_2: $length = 3$, $parent = 4$
- p_3: $length = 5$, $parent = 5$
- p_4: $length = 2$, $parent = 1$
- p_5: $length = 4$, $parent = 4$

Synchronous Bellman-Ford algorithm

In each round,

$$(length\ of\ p_i) = \min_j\{(length\ of\ p_j) + weight_{j,i}\}.$$

The shortest paths are found after $n-1$ rounds.



Round 4:

- p_1: $length = 0$, $parent = \perp$
- p_2: $length = 3$, $parent = 4$
- p_3: $length = 5$, $parent = 5$
- p_4: $length = 2$, $parent = 1$
- p_5: $length = 4$, $parent = 4$

Synchronous Bellman-Ford algorithm

Algorithm Single source shortest path: synchronous Bellman-Ford

```
1:  (Local variables)
2:  length ← ∞
3:  parent ← ⊥
4:  Neighbors ← set of neighbors
5:  {weight_{i,j}, weight_{j,i} | j ∈ Neighbors} ← weights of incident links
6:  if i = i_0 then length := 0
7:  for round := 1 to n − 1 do
8:      Send (i, length) to all p_j for j ∈ Neighbors
9:      Receive (j, length_j) from p_j for each j ∈ Neighbors
10:     for each j ∈ Neighbors do
11:         if length > length_j + weight_{j,i} then
12:             length := length_j + weight_{j,i}
13:             parent := j
14:         end if
15:     end for
16: end for
```

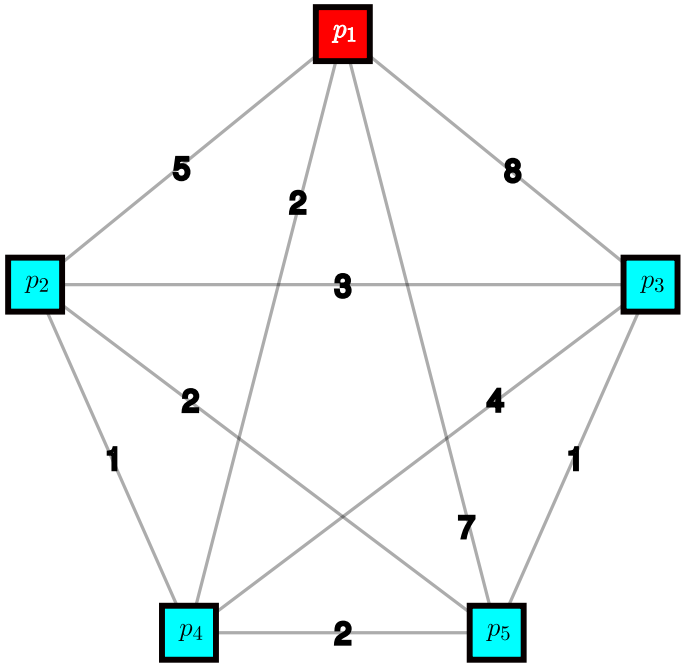
Asynchronous Bellman-Ford algorithm

Messages received:

$$p_1 \xrightarrow{0} p_2, p_3, p_5$$

Messages not received:

$$p_1 \xrightarrow{0} p_4.$$



Process	p_1	p_2	p_3	p_4	p_5
length	0	5	8	∞	7
parent	$\perp$	1	1	$\perp$	1

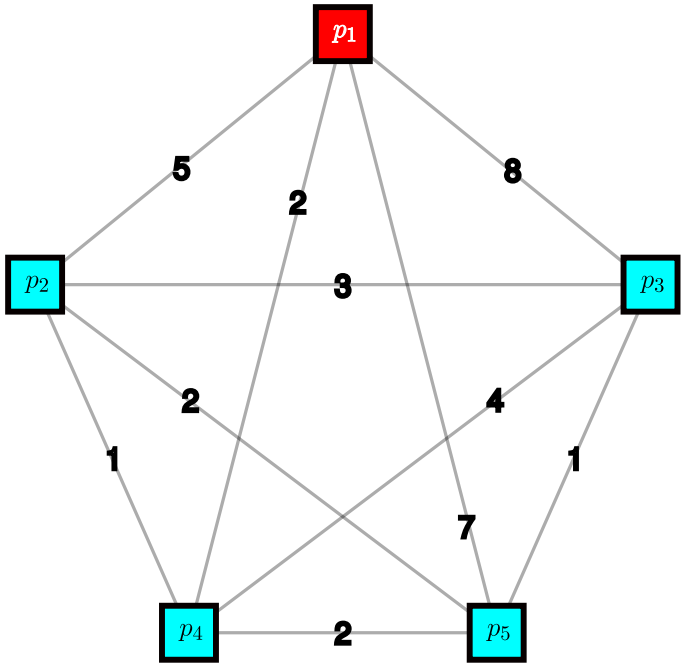
Asynchronous Bellman-Ford algorithm

Messages received:

$$p_2 \xrightarrow{5} p_1, p_3, p_5,$$
$$p_3 \xrightarrow{8} p_1, p_2, p_5,$$
$$p_5 \xrightarrow{7} p_1, p_2, p_3, p_4.$$

Messages not received:

$$p_1 \xrightarrow{0} p_4, \quad p_2 \xrightarrow{5} p_4, \quad p_3 \xrightarrow{8} p_4.$$



Process	p_1	p_2	p_3	p_4	p_5
length	0	5	8	9	7
parent	$\perp$	1	1	5	1

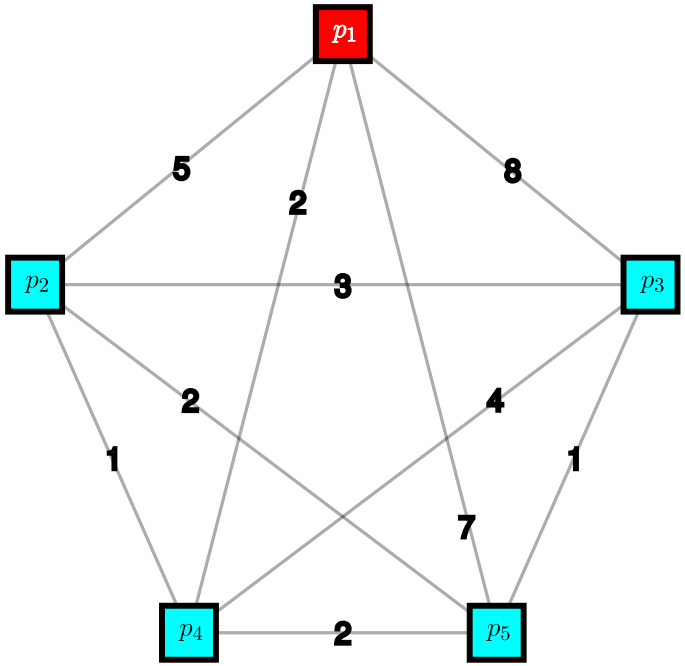
Asynchronous Bellman-Ford algorithm

Messages received:

$$p_4 \xrightarrow{9} p_1, p_2, p_3, p_5,$$
$$p_3 \xrightarrow{8} p_4, \quad p_2 \xrightarrow{5} p_4.$$

Messages not received:

$$p_1 \xrightarrow{0} p_4.$$



Process	p_1	p_2	p_3	p_4	p_5
length	0	5	8	6	7
parent	$\perp$	1	1	2	1

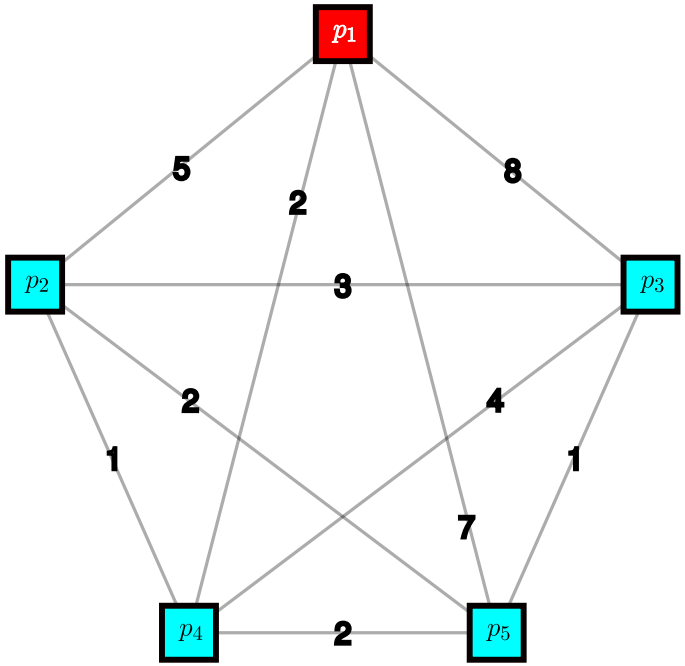
Asynchronous Bellman-Ford algorithm

Messages received:

$$p_4 \xrightarrow{6} p_1, p_2, p_3,$$
$$p_1 \xrightarrow{0} p_4.$$

Messages not received:

$$p_4 \xrightarrow{6} p_5.$$



Process	p_1	p_2	p_3	p_4	p_5
length	0	5	8	2	7
parent	$\perp$	1	1	1	1

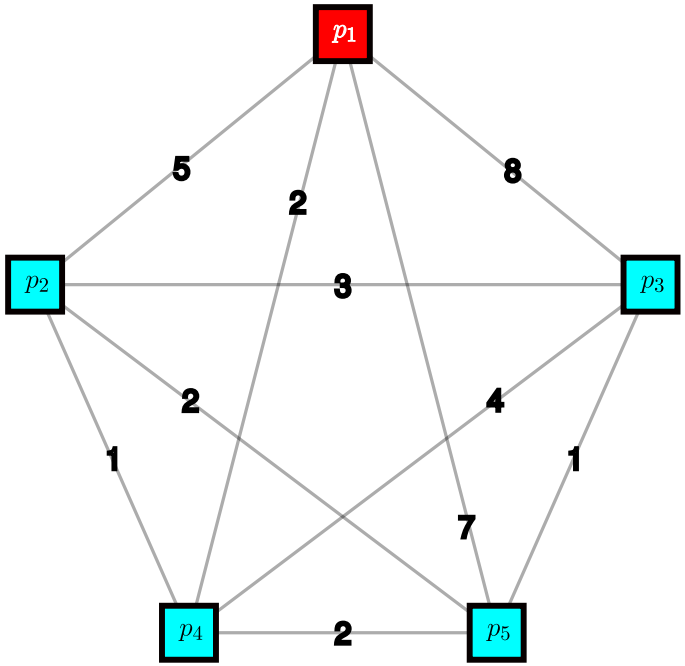
Asynchronous Bellman-Ford algorithm

Messages received:

$$p_4 \xrightarrow{6} p_5,$$
$$p_4 \xrightarrow{2} p_1, p_2, p_3,$$
$$p_1 \xrightarrow{0} p_4.$$

Messages not received:

$$p_4 \xrightarrow{2} p_5$$



Process	p_1	p_2	p_3	p_4	p_5
length	0	3	6	2	7
parent	$\perp$	4	4	1	1

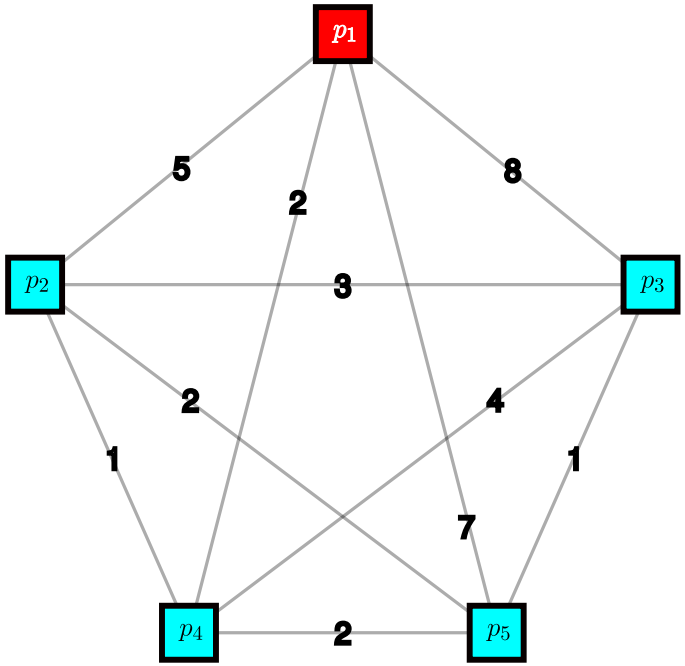
Asynchronous Bellman-Ford algorithm

Messages received:

$$p_2 \xrightarrow{3} p_1, p_3, p_4, p_5,$$
$$p_3 \xrightarrow{2} p_1, p_2, p_4, p_5.$$

Messages not received:

$$p_4 \xrightarrow{2} p_5$$



Process	p_1	p_2	p_3	p_4	p_5
length	0	3	6	2	5
parent	$\perp$	4	4	1	2

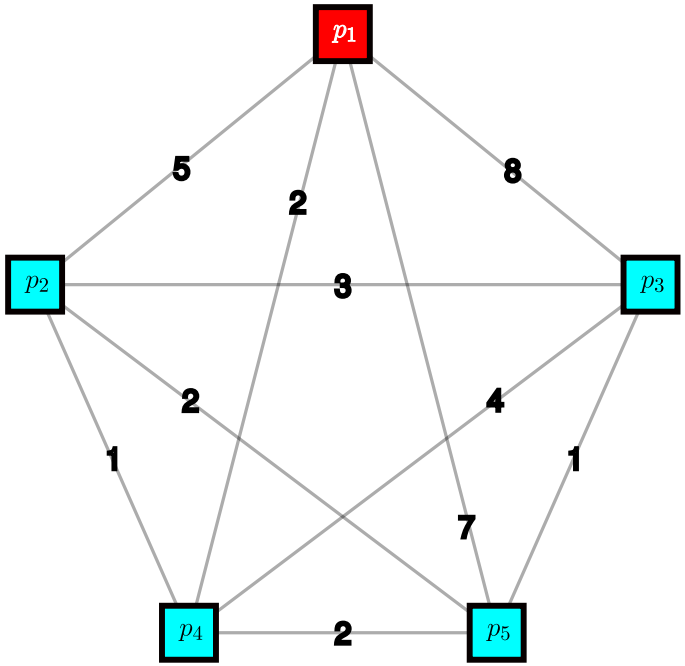
Asynchronous Bellman-Ford algorithm

Messages received:

$$p_5 \xrightarrow{5} p_1, p_2, p_4,$$
$$p_4 \xrightarrow{2} p_5.$$

Messages not received:

$$p_5 \xrightarrow{5} p_3$$

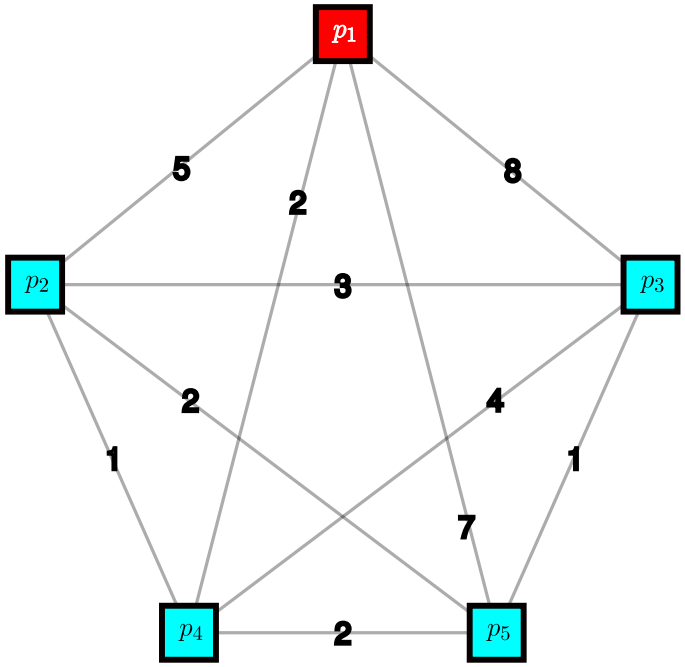


Process	p_1	p_2	p_3	p_4	p_5
length	0	3	6	2	4
parent	$\perp$	4	4	1	4

Asynchronous Bellman-Ford algorithm

Messages received:

$$p_5 \xrightarrow{5} p_3, \quad p_5 \xrightarrow{4} p_1, p_2, p_3, p_4.$$



Process	p_1	p_2	p_3	p_4	p_5
length	0	3	5	2	4
parent	$\perp$	4	5	1	4

Asynchronous Bellman-Ford algorithm

Algorithm Single source shortest path: asynchronous Bellman-Ford

```
1:  (Local variables)
2:  length ← ∞, parent ← ⊥
3:  Neighbors ← set of neighbors
4:  {weight_{i,j}, weight_{j,i} | j ∈ Neighbors} ← weights of incident links

5:  if i = i_0 then
6:      length := 0
7:      Send (i_0, 0) to p_j for all j ∈ Neighbors
8:      Terminate
9:  end if

10: on receiving (j, length_j) do
11:     if length > length_j + weight_{j,i} then
12:         length := length_j + weight_{j,i}
13:         parent := j
14:         Send (i, length) to p_j for all j ∈ Neighbors
15:     end if
16: end on
```

On synchronous and asynchronous algorithms

Synchronous Bellman-Ford	Asynchronous Bellman-Ford
Terminate after $(n-1)$ rounds	No termination condition[1]
Message complexity: $O(nl)$	Worse-case complexity: $\Omega(c^n)$
Not fault-tolerant	Works when some nodes are faulty

In general,

- It is much easier to design synchronous algorithms.
- In failure-free asynchronous systems, we may want to transform synchronous algorithms to asynchronous algorithms using synchronizers.
- Primitive asynchronous algorithms are usually more efficient than transformed algorithms.

[1] For an improved version, see "K. M. Chandy and J. Misra, Distributed Computation on Graphs: Shortest Path Algorithms, *Communications of the ACM*, **25**(11), 1982, 833–838."

1 Broadcast

2 Shortest-path problem

3 Synchronizers

4 Summary

Synchronizers

- In an asynchronous system, a synchronizer is a mechanism that indicates to each process when it is safe to proceed to the next round of execution of the "synchronous" algorithm.

Synchronizers

- In an asynchronous system, a synchronizer is a mechanism that indicates to each process when it is safe to proceed to the next round of execution of the "synchronous" algorithm.
- To transform a synchronous algorithm to an asynchronous algorithm, simply add a synchronizer after each round of execution.

Synchronizers

- In an asynchronous system, a synchronizer is a mechanism that indicates to each process when it is safe to proceed to the next round of execution of the "synchronous" algorithm.
- To transform a synchronous algorithm to an asynchronous algorithm, simply add a synchronizer after each round of execution.
- Using synchronizers might increase the message complexity and total execution time, but can significantly simplify the design of algorithms.

The simple synchronizer

- **Assumption:** In each round, each process sends one and only one message to all neighbors. If not, combine multiple messages into one or send a dummy message.
- **Synchronizer:** A process moves to the next round only after it receives messages from all neighbors.
- **Properties:**
  1. In physical time, any two adjacent processes may be only one round apart.
  2. When process i is in round $round_i$, it can receive messages only from rounds $round_i$ or $round_i + 1$ from its neighbors.

Algorithm for the simple synchronizer

Algorithm Simple synchronizer

```
1:  (Local variables)
2:  Neighbors ← set of neighbors
3:  RecvCurrRound ← {}, RecvNextRound ← {}

4:  on receiving a message (round, msg) from p_j in Round k do
5:      Do something
6:      if round = k then
7:          RecvCurrRound := RecvCurrRound ∪ {j}
8:      else
9:          RecvNextRound := RecvNextRound ∪ {j}
10:     end if
11:     if RecvCurrRound = Neighbors then
12:         RecvCurrRound := RecvNextRound
13:         RecvNextRound := {}
14:         Move to the next round
15:     end if
16: end on
```

The α Synchronizer

We now remove the assumption on the number of messages sent between neighbors in each round.

The α Synchronizer

We now remove the assumption on the number of messages sent between neighbors in each round.

- **Q:** When can a process p_i move to the next round?

The α Synchronizer

We now remove the assumption on the number of messages sent between neighbors in each round.

- **Q:** When can a process p_i move to the next round?
  A: After p_i receives all messages sent to it.

The α Synchronizer

We now remove the assumption on the number of messages sent between neighbors in each round.

- **Q:** When can a process p_i move to the next round?
  A: After p_i receives all messages sent to it.
- **Q:** How does p_i know if it has received all messages?

The α Synchronizer

We now remove the assumption on the number of messages sent between neighbors in each round.

- **Q:** When can a process p_i move to the next round?
  A: After p_i receives all messages sent to it.
- **Q:** How does p_i know if it has received all messages?
  A: For each neighbor p_j, it should send p_i an extra message indicating that all messages that p_j sent to p_i have been received.

The α Synchronizer

We now remove the assumption on the number of messages sent between neighbors in each round.

- **Q:** When can a process p_i move to the next round?
  A: After p_i receives all messages sent to it.
- **Q:** How does p_i know if it has received all messages?
  A: For each neighbor p_j, it should send p_i an extra message indicating that all messages that p_j sent to p_i have been received.
- **Q:** How does p_j know if p_i has received a message that p_j sent?

The α Synchronizer

We now remove the assumption on the number of messages sent between neighbors in each round.

- **Q:** When can a process p_i move to the next round?
  A: After p_i receives all messages sent to it.
- **Q:** How does p_i know if it has received all messages?
  A: For each neighbor p_j, it should send p_i an extra message indicating that all messages that p_j sent to p_i have been received.
- **Q:** How does p_j know if p_i has received a message that p_j sent?
  A: p_i needs to acknowledge each message sent by p_j through an "ack" message sent back to p_j.

The α Synchronizer

We now remove the assumption on the number of messages sent between neighbors in each round.

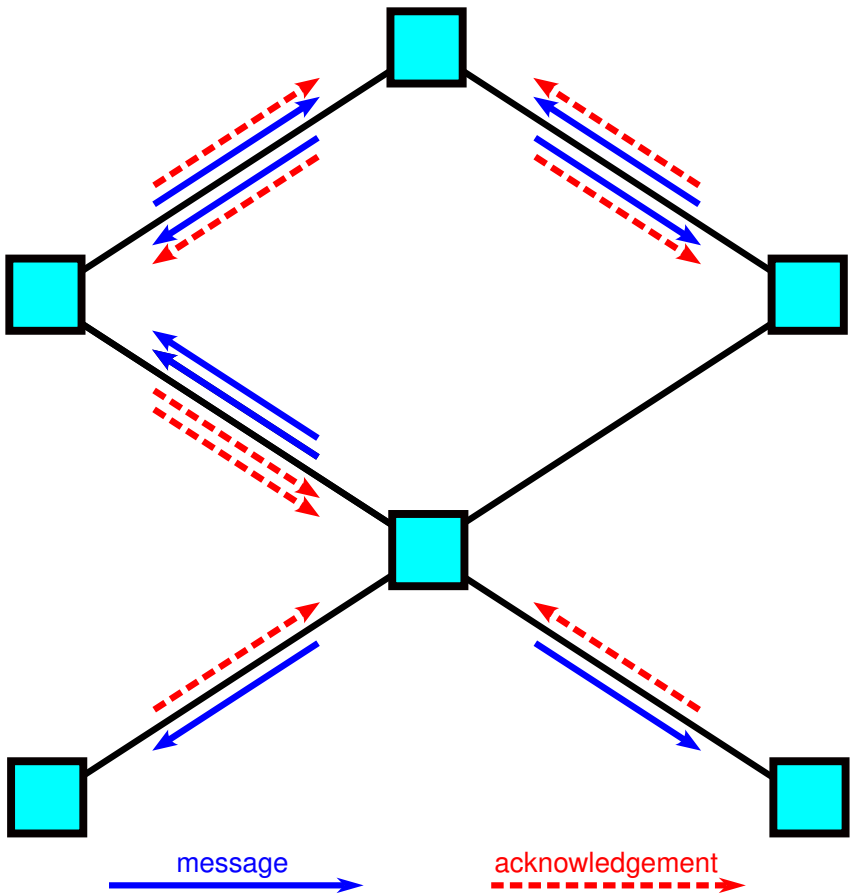
- **Q:** When can a process p_i move to the next round?
  A: After p_i receives all messages sent to it.
- **Q:** How does p_i know if it has received all messages?
  A: For each neighbor p_j, it should send p_i an extra message indicating that all messages that p_j sent to p_i have been received.
- **Q:** How does p_j know if p_i has received a message that p_j sent?
  A: p_i needs to acknowledge each message sent by p_j through an "ack" message sent back to p_j.

Summary:

- A process p_j is said to be safe in round r if all messages sent by p_j in round r have been acknowledged.
- A process p_i can proceed to Round $r+1$ if all its neighbors are safe in round r.

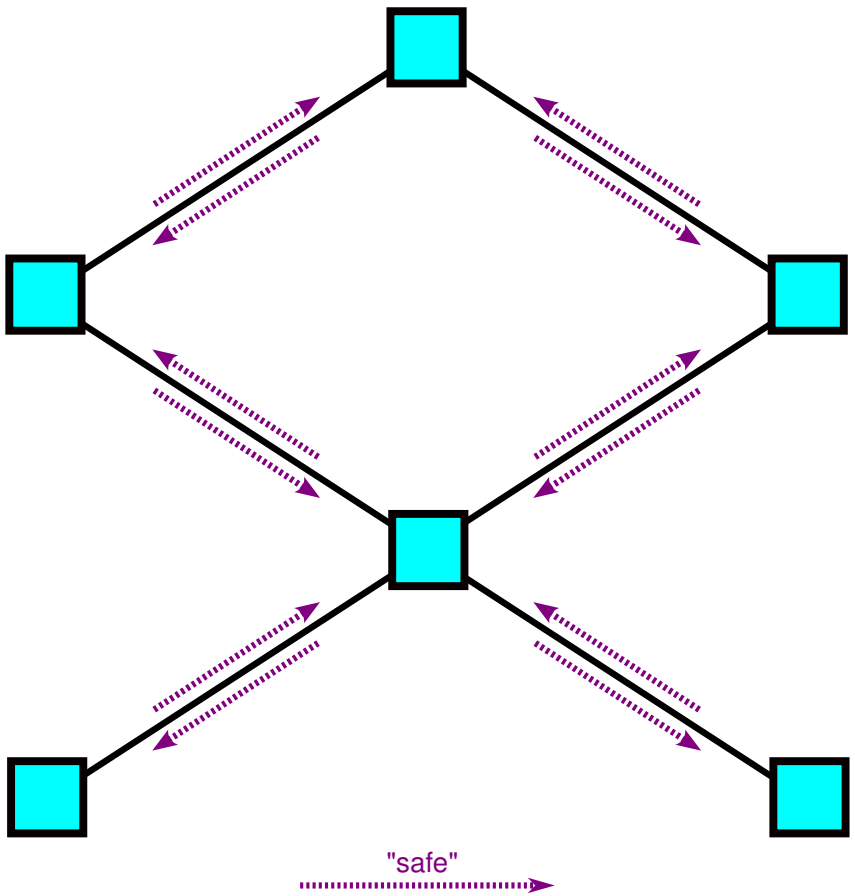
Example of the α Synchronizer

1. Send messages and acknowledgements:



Example of the α Synchronizer

2. Inform neighbors that "I'm safe".



1 Broadcast

2 Shortest-path problem

3 Synchronizers

4 Summary

Summary

Summary:

- Broadcast and four types of broadcasts. Implementation of FIFO and causal broadcasts.
- Synchronous and asynchronous Bellman-Ford algorithms.
- The simple and α synchronizers.